

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving enjoysport.csv to enjoysport.csv

```
import pandas as pd
```

```
data = pd.read_csv("enjoysport.csv")
```

```
print(data)
```

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Strong	Cool	Change	Yes

```
import pandas as pd
```

```
data = pd.read_csv("enjoysport.csv")
```

```
concepts = data.iloc[:, :-1].values
```

```
target = data.iloc[:, -1].values
```

```
hypothesis = concepts[0].copy()
```

```
print("Initial Hypothesis:")
```

```
print(hypothesis)
```

```
for i in range(len(concepts)):
```

```
    if target[i].lower() == "yes":
```

```
        for j in range(len(hypothesis)):
```

```
            if hypothesis[j] != concepts[i][j]:
```

```
                hypothesis[j] = '?'
```

```
        print("\nAfter Example", i+1)
```

```
        print(hypothesis)
```

```
print("\nFinal Hypothesis:")
```

```
print(hypothesis)
```

Initial Hypothesis:

```
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
```

After Example 1

```
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
```

After Example 2

```
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
```

After Example 4

```
['Sunny' 'Warm' '?' 'Strong' '?' '?']
```

Final Hypothesis:

```
['Sunny' 'Warm' '?' 'Strong' '?' '?']
```

Breast Cancer Prediction using Machine Learning

Step 1: Import libraries

```
import pandas as pd
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load dataset

```
data = load_breast_cancer()
```

```
X = data.data      # input features
```

```

y = data.target      # output class

# Step 3: Convert to DataFrame
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

print("Dataset Sample:")
print(df.head())

print("\nTarget Names:")
print(data.target_names)
# 0 = malignant, 1 = benign

# Step 4: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Train ML model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Step 7: Prediction
y_pred = model.predict(X_test)

# Step 8: Evaluation
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=data.target_names))

# Step 9: Predict one new sample
sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)

print("\nSample Prediction:")
if prediction[0] == 0:
    print("Malignant Breast Cancer")
else:
    print("Benign / Non-Cancer")

```

```

Dataset Sample:
mean radius mean texture mean perimeter mean area mean smoothness \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38      77.58      386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

mean compactness mean concavity mean concave points mean symmetry \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
4      0.13280      0.1980      0.10430      0.1809

mean fractal dimension ... worst texture worst perimeter worst area \
0      0.07871 ...      17.33      184.60      2019.0
1      0.05667 ...      23.41      158.80      1956.0
2      0.05999 ...      25.53      152.50      1709.0
3      0.09744 ...      26.50      98.87      567.7
4      0.05883 ...      16.67      152.20      1575.0

worst smoothness worst compactness worst concavity worst concave points \
0      0.1622      0.6656      0.7119      0.2654
1      0.1238      0.1866      0.2416      0.1860
2      0.1444      0.4245      0.4504      0.2430
3      0.2098      0.8663      0.6869      0.2575
4      0.1374      0.2050      0.4000      0.1625

worst symmetry worst fractal dimension target

```

```

0      0.4601      0.11890      0
1      0.2750      0.08902      0
2      0.3613      0.08758      0
3      0.6638      0.17300      0
4      0.2364      0.07678      0

```

[5 rows x 31 columns]

Target Names:
['malignant' 'benign']

Accuracy:
0.9736842105263158

Confusion Matrix:
[[41 2]
 [1 70]]

Classification Report:

	precision	recall	f1-score	support
malignant	0.98	0.95	0.96	43
benign	0.97	0.99	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

data = pd.read_csv("Iris.csv")

print(data.head())
print(data.shape)

# Remove ID column if present
if "Id" in data.columns:
    data = data.drop("Id", axis=1)

target_column = "Species"

label_encoder = LabelEncoder()
data[target_column] = label_encoder.fit_transform(data[target_column])

X = data.drop(target_column, axis=1)
y = data[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("\nModel:", name)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print(classification_report(y_test, y_pred))

```

```

-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_894/926120030.py in <cell line: 0>()
      7 from sklearn.naive_bayes import GaussianNB
      8
----> 9 data = pd.read_csv("Iris.csv")
     10
     11 print(data.head())

----- 4 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
     871         if ioargs.encoding and "b" not in ioargs.mode:
     872             # Encoding
--> 873             handle = open(
     874                 handle,
     875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'Iris.csv'

```

```

import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

```

```
data = load_breast_cancer()
```

```
X = data.data
y = data.target
```

```
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y
```

```
print("Dataset Sample:")
print(df.head())
```

```
print("\nTarget Names:")
print(data.target_names)
# 0 = malignant, 1 = benign
```

```

Dataset Sample:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38      77.58      386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
4      0.13280      0.1980      0.10430      0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0      0.07871  ...      17.33      184.60      2019.0
1      0.05667  ...      23.41      158.80      1956.0
2      0.05999  ...      25.53      152.50      1709.0
3      0.09744  ...      26.50      98.87      567.7
4      0.05883  ...      16.67      152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0      0.1622      0.6656      0.7119      0.2654
1      0.1238      0.1866      0.2416      0.1860
2      0.1444      0.4245      0.4504      0.2430
3      0.2098      0.8663      0.6869      0.2575
4      0.1374      0.2050      0.4000      0.1625

   worst symmetry  worst fractal dimension  target
0      0.4601      0.11890      0
1      0.2750      0.08902      0
2      0.3613      0.08758      0
3      0.6638      0.17300      0

```

```
4          0.2364          0.07678          0

[5 rows x 31 columns]

Target Names:
['malignant' 'benign']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

```
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

```
Accuracy:
0.9736842105263158
```

```
Confusion Matrix:
[[41  2]
 [ 1 70]]
```

```
Classification Report:
              precision    recall  f1-score   support

   malignant       0.98       0.95       0.96         43
     benign       0.97       0.99       0.98         71

   accuracy              0.97              114
  macro avg              0.97              114
 weighted avg              0.97              114
```

```
sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)

print("\nSample Prediction:")
if prediction[0] == 0:
    print("Malignant Breast Cancer")
else:
    print("Benign / Non-Cancer")
```

```
Sample Prediction:
Benign / Non-Cancer
```

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
iris = load_iris()

X = iris.data
y = iris.target
```

```
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y

print("Dataset Sample:")
print(df.head())

print("\nTarget Classes:")
print(iris.target_names)
# 0 = Setosa
# 1 = Versicolor
# 2 = Virginica
```

```
Dataset Sample:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2
```

```
   target
0       0
1       0
2       0
3       0
4       0
```

```
Target Classes:
['setosa' 'versicolor' 'virginica']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=200)
```

```
y_pred = model.predict(X_test)
```

```
print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

```
Accuracy:
```

```
1.0
```

```
Confusion Matrix:
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)
```

```
print("\nSample Prediction:")
```

```
if prediction[0] == 0:
    print("Setosa")
elif prediction[0] == 1:
    print("Versicolor")
else:
    print("Virginica")
```

```
Sample Prediction:
Versicolor
```